

“FOOD STORE”- SISTEMA DE GESTIÓN DE PEDIDO DE COMIDA

Tecnicatura Universitaria en Programación a Distancia – Programación III
Junio 2026

Alderete Daniel
Alfredo Oscar

Índice

1. Introducción	2
2. Marco Teórico	3
3. Estructura del Proyecto	5
4. Decisiones Técnicas y Arquitectura	7
5. Capturas de Pantalla	9
6. Dificultades y Soluciones	18
7. Bibliografía / Webgrafía	20

1. INTRODUCCIÓN

El presente trabajo práctico integrador consiste en el desarrollo de una aplicación web full stack denominada **Food Store**, orientada a la simulación de un sistema de ecommerce de comidas. El proyecto integra los conocimientos adquiridos durante la cursada de Programación III, aplicando tecnologías de desarrollo frontend y backend en un caso de uso real.

El sistema contempla dos perfiles de usuario: el **Administrador**, quien gestiona el catálogo de productos, categorías y el estado de los pedidos; y el **Cliente**, quien puede registrarse, navegar el catálogo, agregar productos al carrito y confirmar compras.

2. MARCO TEÓRICO

Java 23

Java es un lenguaje de programación orientado a objetos, de propósito general y fuertemente tipado. En este proyecto se utilizó la versión 23, que incorpora mejoras en el rendimiento y nuevas características del lenguaje como pattern matching y records. Java fue elegido para el desarrollo del backend por su robustez, amplia documentación y compatibilidad con el ecosistema JPA/Hibernate.

JPA (Jakarta Persistence API)

JPA es una especificación de Java que define una interfaz estándar para el mapeo objeto-relacional (ORM). Permite trabajar con bases de datos relacionales utilizando objetos Java, eliminando la necesidad de escribir SQL directamente. En este proyecto se utilizó la versión 3.1, compatible con Jakarta EE.

Hibernate

Hibernate es la implementación más utilizada de JPA. Actúa como proveedor de persistencia y se encarga de traducir las operaciones sobre objetos Java en consultas SQL hacia la base de datos. Se utilizó la versión 6.4.4. Final, que ofrece soporte completo para Jakarta Persistence 3.1 y mejoras en el manejo de tipos y consultas JPQL.

H2 Database

H2 es una base de datos relacional escrita en Java, liviana y de fácil configuración. Se utilizó en modo archivo (jdbc:h2:file:./data/jpa_db), lo que permite que los datos persistan entre ejecuciones del sistema. Es ideal para entornos de desarrollo y proyectos educativos por su simplicidad de configuración y compatibilidad con Hibernate.

Gradle

Gradle es una herramienta de automatización de construcción de proyectos. Se utilizó con el DSL Groovy para gestionar las dependencias del backend (Hibernate, H2, Jakarta Persistence API) y compilar el proyecto Java.

TypeScript

TypeScript es un superconjunto tipado de JavaScript desarrollado por Microsoft. Agrega tipado estático opcional al lenguaje, lo que permite detectar errores en tiempo de compilación y mejorar la mantenibilidad del código. En este proyecto se utilizó para desarrollar toda la lógica del frontend, desde la autenticación hasta el manejo del carrito.

Vite

Vite es una herramienta de construcción y servidor de desarrollo para proyectos frontend modernos. Ofrece arranque instantáneo del servidor de desarrollo gracias al uso de módulos ES nativos del navegador, y hot module replacement (HMR) para actualizar los cambios en tiempo real sin recargar la página. Se utilizó la versión 8 con la plantilla vanilla-ts.

HTML5 y CSS3

HTML5 es el lenguaje de marcado estándar para la estructuración de páginas web. CSS3 es el lenguaje de estilos utilizado para definir la presentación visual del sistema. En este proyecto se utilizaron sin frameworks adicionales, aplicando diseño responsive y variables CSS para mantener consistencia visual en todas las páginas.

localStorage

localStorage es una API del navegador que permite almacenar datos de forma persistente en el cliente, sin fecha de vencimiento. En este proyecto se utilizó para simular la autenticación de usuarios, persistir el carrito de compras entre sesiones y almacenar los pedidos y cambios realizados por

el administrador. Su uso es exclusivamente educativo y no representa una práctica recomendada para entornos de producción.

3. ESTRUCTURA DEL PROYECTO

El proyecto está organizado en dos módulos independientes:

Frontend ([final-prog3](#))

final-prog3/

```
├── index.html      # Redirección automática al login
├── package.json    # Dependencias y scripts
├── tsconfig.json   # Configuración TypeScript
├── vite.config.ts  # Configuración Vite
└── src/
    ├── main.ts     # Punto de entrada
    ├── style.css    # Estilos globales
    ├── types/
    |   └── index.ts # Interfaces TypeScript
    ├── utils/
    |   ├── auth.ts  # Manejo de sesión
    |   └── carrito.ts # Manejo del carrito
    ├── data/
    |   ├── usuarios.json
    |   ├── categorias.json
    |   ├── productos.json
    |   └── pedidos.json
    └── pages/
        ├── auth/login/
        └── auth/register/
```

```
└─ store/home/
└─ store/productDetail/
└─ store/cart/
└─ client/orders/
└─ admin/
    └─ adminHome/
    └─ categories/
    └─ products/
    └─ orders/
```

Backend (foodstore-backend)

```
foodstore-backend/
└─ build.gradle
└─ settings.gradle
└─ src/main/
    └─ java/com/tp/jpa/
        | └─ Main.java
        | └─ model/
        |     | └─ Base.java
        |     | └─ Calculable.java
        |     | └─ Categoria.java
        |     | └─ Producto.java
        |     | └─ Usuario.java
        |     | └─ Pedido.java
        |     | └─ DetallePedido.java
        |     └─ enums/
```

```

| | └─ Rol.java
| | └─ Estado.java
| | └─ FormaPago.java
| └─ repository/
| | └─ BaseRepository.java
| | └─ CategoriaRepository.java
| | └─ ProductoRepository.java
| | └─ UsuarioRepository.java
| | └─ PedidoRepository.java
| └─ util/
|   └─ JPAUtil.java
└─ resources/
    └─ META-INF/
        └─ persistence.xml

```

4. DECISIONES TÉCNICAS Y ARQUITECTURA

Patrón Repository

Se implementó el patrón Repository para abstraer el acceso a la base de datos. La clase `BaseRepository<T>` es genérica y provee las operaciones CRUD comunes (guardar, buscarPorId, listarActivos, eliminarLogico) para todas las entidades. Cada repositorio específico extiende esta clase base y agrega consultas JPQL propias según las necesidades del dominio.

Esta decisión permite separar la lógica de negocio de la lógica de persistencia, facilitando el mantenimiento y la extensibilidad del sistema.

Baja Lógica

En lugar de eliminar físicamente los registros de la base de datos, se implementó la baja lógica mediante el campo eliminado en la clase Base. Cuando se da de baja una entidad, se establece `eliminado = true` y el

registro permanece en la base de datos. Todos los listados filtran por `eliminado = false`, garantizando que los datos históricos no se pierdan.

Interfaz Calculable

Se definió la interfaz `Calculable` con el método `calcularTotal():void`, implementado por la entidad `Pedido`. Este método suma los subtotales de todos los `DetallePedido` asociados y asigna el resultado al campo `total` del pedido. Esta decisión sigue el principio de responsabilidad única, delegando el cálculo del total a la propia entidad.

Transacción Atómica en Alta de Pedido

El alta de pedido es la operación más compleja del sistema. Se implementó dentro de una única transacción JPA que incluye la creación del pedido, la generación de los detalles, el cálculo del total y la reducción del stock de cada producto. Si cualquier operación falla, se realiza un `rollback` completo, garantizando la integridad de los datos.

Arquitectura Frontend sin Framework

Se decidió no utilizar frameworks como `React` o `Vue` para el frontend, siguiendo las indicaciones de la consigna (`TypeScript` vanilla + `Vite`). Cada página es un módulo independiente con su propio `HTML`, `CSS` y `TypeScript`. La comunicación entre páginas se realiza mediante parámetros en la URL (para el detalle de producto) y `localStorage` (para la sesión y el carrito).

Persistencia Frontend con localStorage

Dado que el frontend no se conecta al backend (son dos sistemas independientes en esta fase 1), se utilizó `localStorage` como mecanismo de persistencia del lado del cliente. Los datos de usuarios, productos, categorías y pedidos se cargan desde archivos `JSON` y los cambios se guardan en `localStorage`, simulando el comportamiento de una API REST.

Autenticación Basada en Roles

El sistema implementa protección de rutas mediante funciones utilitarias (`requiereLogin`, `requiereAdmin`) que verifican la sesión almacenada en `localStorage` antes de renderizar cada página. Si el usuario no tiene sesión activa o no tiene el rol requerido, es redirigido automáticamente al login o al catálogo según corresponda.

5. CAPTURAS DE PANTALLA

- **Pantalla de Login** — formulario de inicio de sesión con las cuentas de prueba



The screenshot shows a mobile application login screen for 'Food Store'. At the top is a yellow burger icon. Below it, the text 'Food Store' is in orange, and 'Iniciar Sesión' is in gray. There are two input fields: 'Email' with the placeholder 'ejemplo@correo.com' and 'Contraseña' with the placeholder 'Ingresá tu contraseña'. Below these is an orange 'Ingresar' button. A link '¿No tenés cuenta? [Registrate aquí](#)' is below the button. At the bottom, under the heading 'Cuentas de prueba:', are the test credentials: 'Admin: admin@admin.com / 123456' and 'Cliente: cliente@food.com / cliente123'.



Food Store

Iniciar Sesión

Email

ejemplo@correo.com

Contraseña

Ingresá tu contraseña

Ingresar

¿No tenés cuenta? [Registrate aquí](#)

Cuentas de prueba:

Admin: admin@admin.com / 123456

Cliente: cliente@food.com / cliente123

- **Pantalla de Registro** — formulario de registro de nuevo usuario



Food Store

Crear Cuenta

Nombre

Apellido

Email

Celular (opcional)

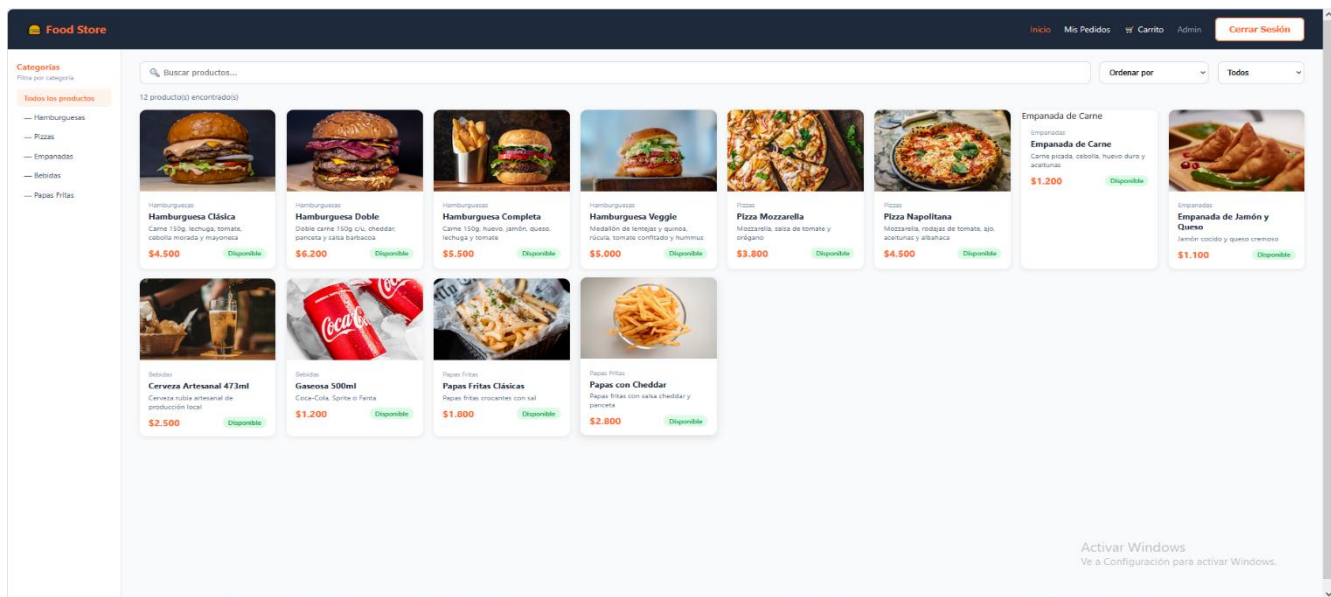
Contraseña

Confirmar Contraseña

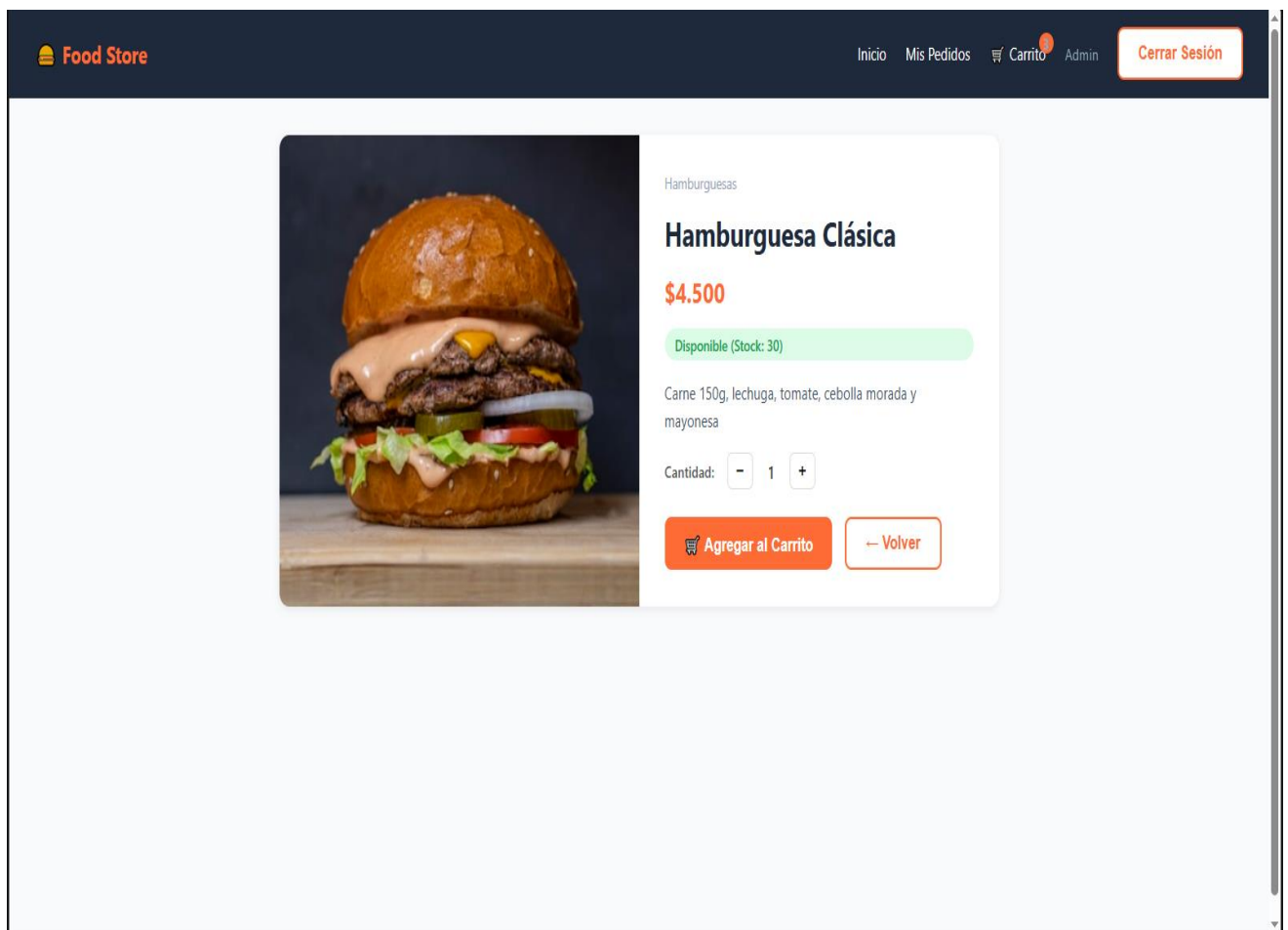
Crear Cuenta

¿Ya tenés cuenta? [Iniciá sesión](#)

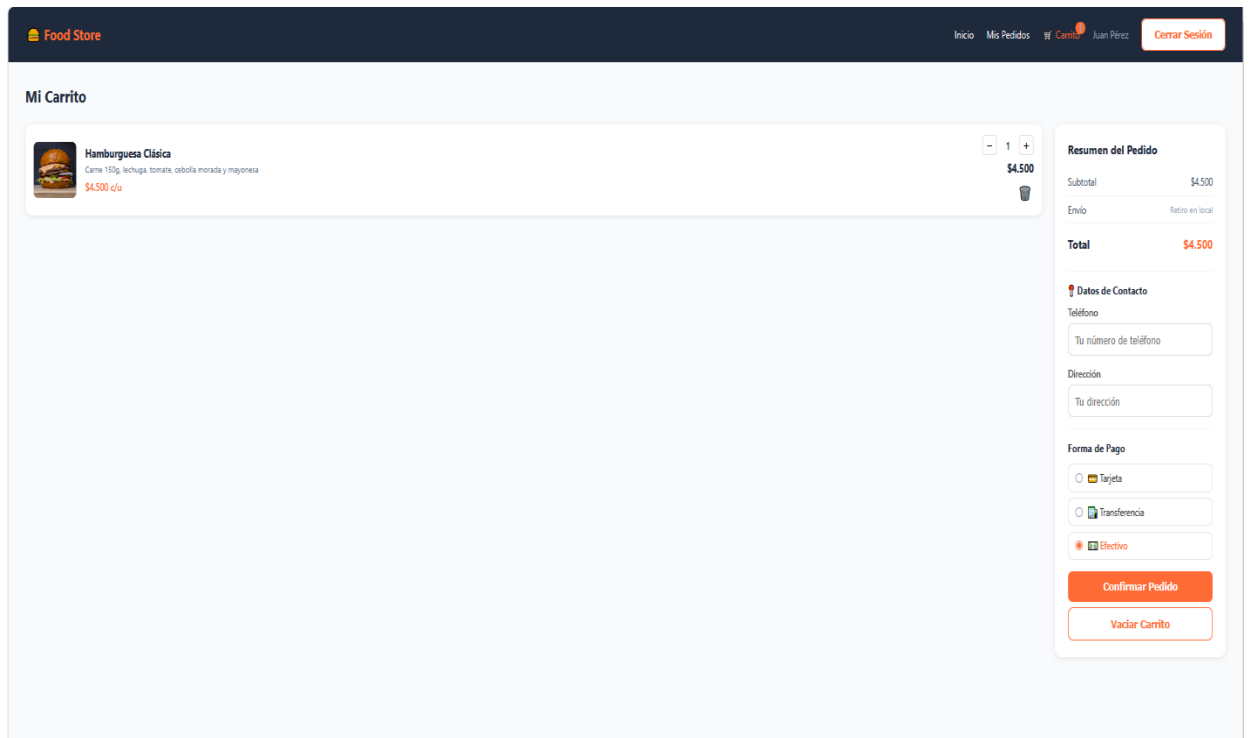
- Catálogo / Home



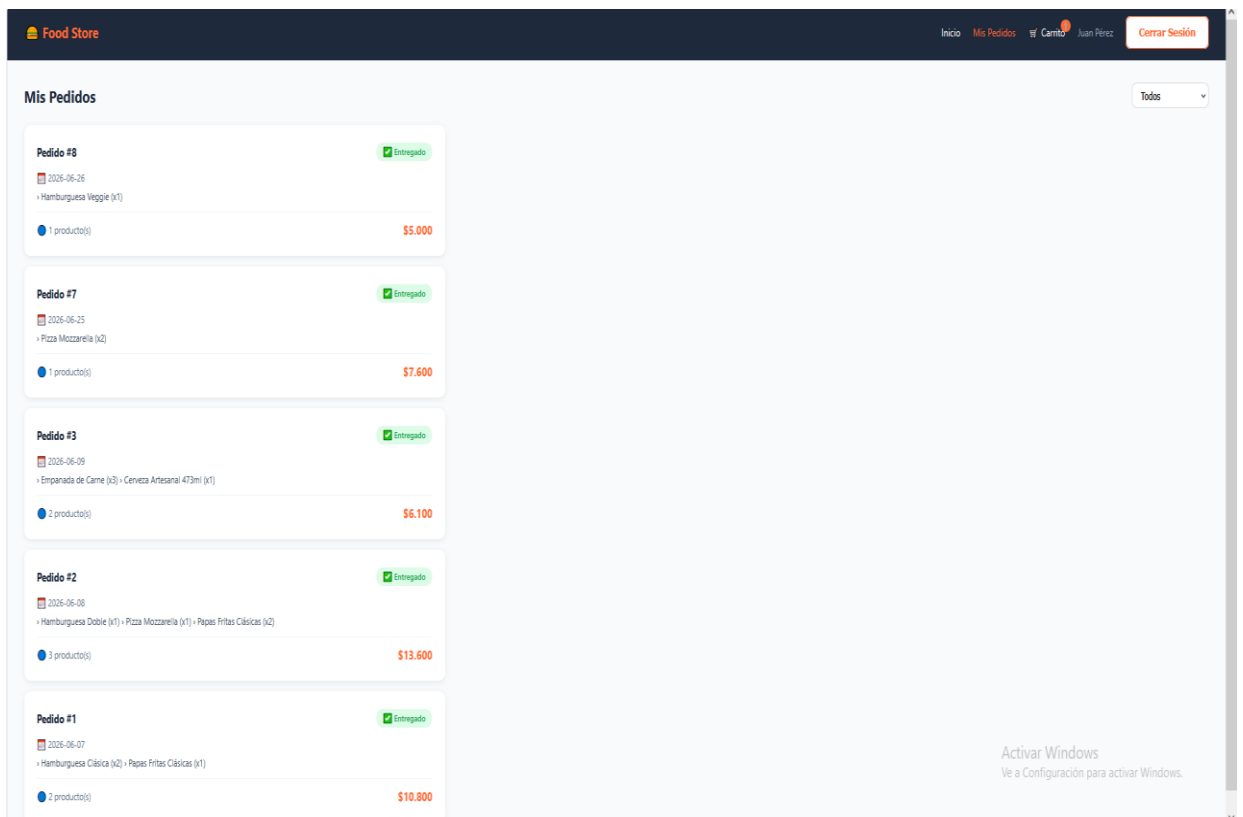
- Detalle de Producto — precio, stock y botón agregar al carrito



- **Carrito** — lista de productos, resumen, datos de contacto y forma de pago



- **Mis Pedidos (Cliente)** — historial de pedidos con badges de estado



- Modal detalle de pedido — información completa del pedido

Pedido #9
×

 **Pendiente**
 2026-06-27

 **Información de Entrega**
 Teléfono: No especificado
 Método de pago:  EFECTIVO

 **Productos**

Hamburguesa Clásica (x1)	\$4.500
Total:	\$4.500

Tu pedido está siendo procesado. Te notificaremos cuando sea confirmado.

- Dashboard Admin

Tienda
 [Panel Admin](#)
 Admin
 [Cerrar Sesión](#)

Administración
 Panel de control
[Dashboard](#)
[Categorías](#)
[Productos](#)
[Pedidos](#)
[Ver Tienda](#)

Panel de Administración


 Categorías
5
[Gestionar](#)


 Productos
12
[Gestionar](#)


 Pedidos
3
[Gestionar](#)

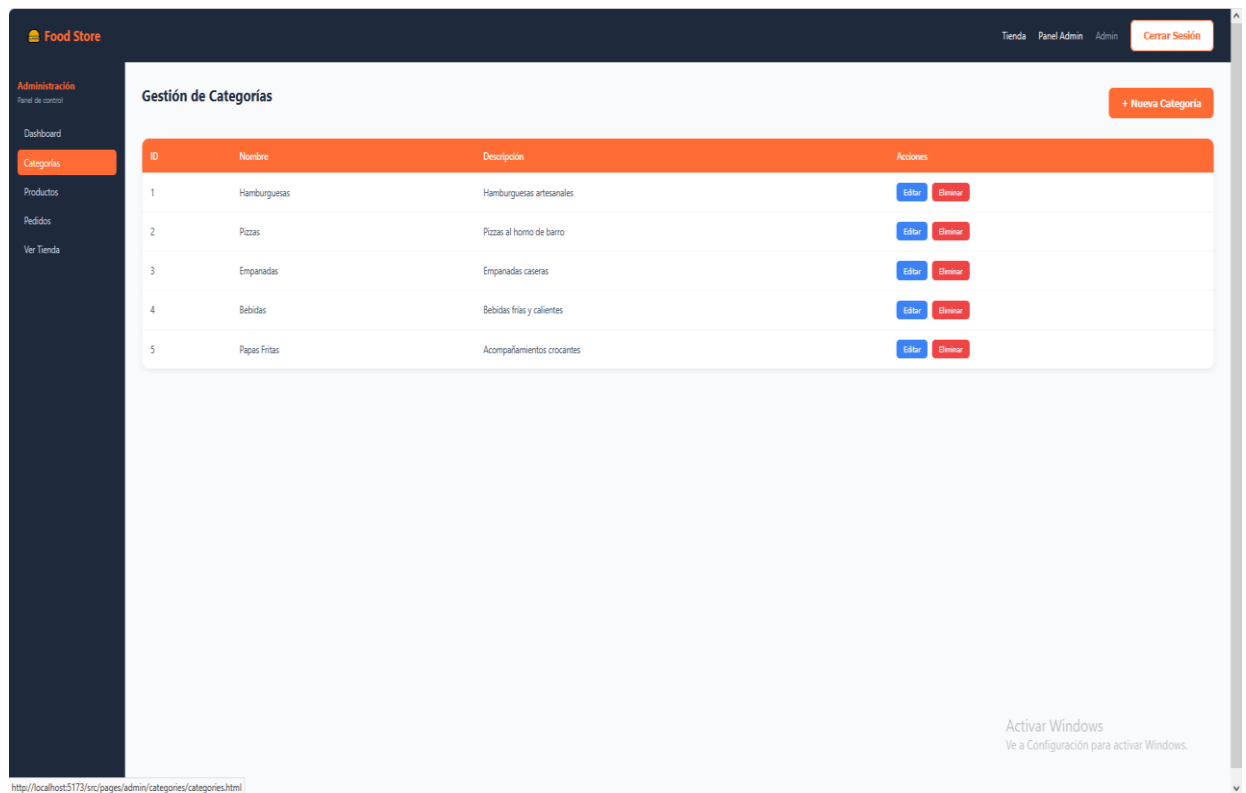

 Disponibles
12
 Productos activos

Resumen Rápido

Ingresos Totales	\$10.800
Pedidos Pendientes	1
En Preparación	1
Completados	1

Activar Windows
 Ve a Configuración para activar Windows.

- **Gestión de Categorías** — tabla con botones editar/eliminar y modal



de alta

- Gestión de Productos — tabla completa con imagen, stock y estado

Food Store

Tienda Panel Admin Admin Cerrar Sesión

Administración

Panel de control

Dashboard

Categorías

Productos

Pedidos

Ver Tienda

Gestión de Productos

+ Nuevo Producto

ID	Imagen	Nombre	Precio	Categoría	Stock	Estado	Acciones
1		Hamburguesa Clásica	\$4.500	Hamburguesas	29	Disponible	Editar Eliminar
2		Hamburguesa Doble	\$6.200	Hamburguesas	40	Disponible	Editar Eliminar
3		Hamburguesa Completa	\$5.500	Hamburguesas	30	Disponible	Editar Eliminar
4		Hamburguesa Veggie	\$5.000	Hamburguesas	19	Disponible	Editar Eliminar
5		Pizza Mozzarella	\$3.800	Pizzas	25	Disponible	Editar Eliminar
6		Pizza Napolitana	\$4.500	Pizzas	25	Disponible	Editar Eliminar
7		Empanada de Carne	\$1.200	Empanadas	50	Disponible	Editar Eliminar
8		Empanada de Jamón y Queso	\$1.100	Empanadas	50	Disponible	Editar Eliminar
9		Cerveza Artesanal 473ml	\$2.500	Bebidas	60	Disponible	Editar Eliminar
10		Gaseosa 500ml	\$1.200	Bebidas	80	Disponible	Editar Eliminar
11		Papas Fritas Clásicas	\$1.800	Papas Fritas	40	Disponible	Editar Eliminar

- Gestión de Pedidos Admin — lista con botón cambiar estado

Food Store

Tienda Panel Admin Admin Cerrar Sesión

Administración

Panel de control

Dashboard

Categorías

Productos

Pedidos

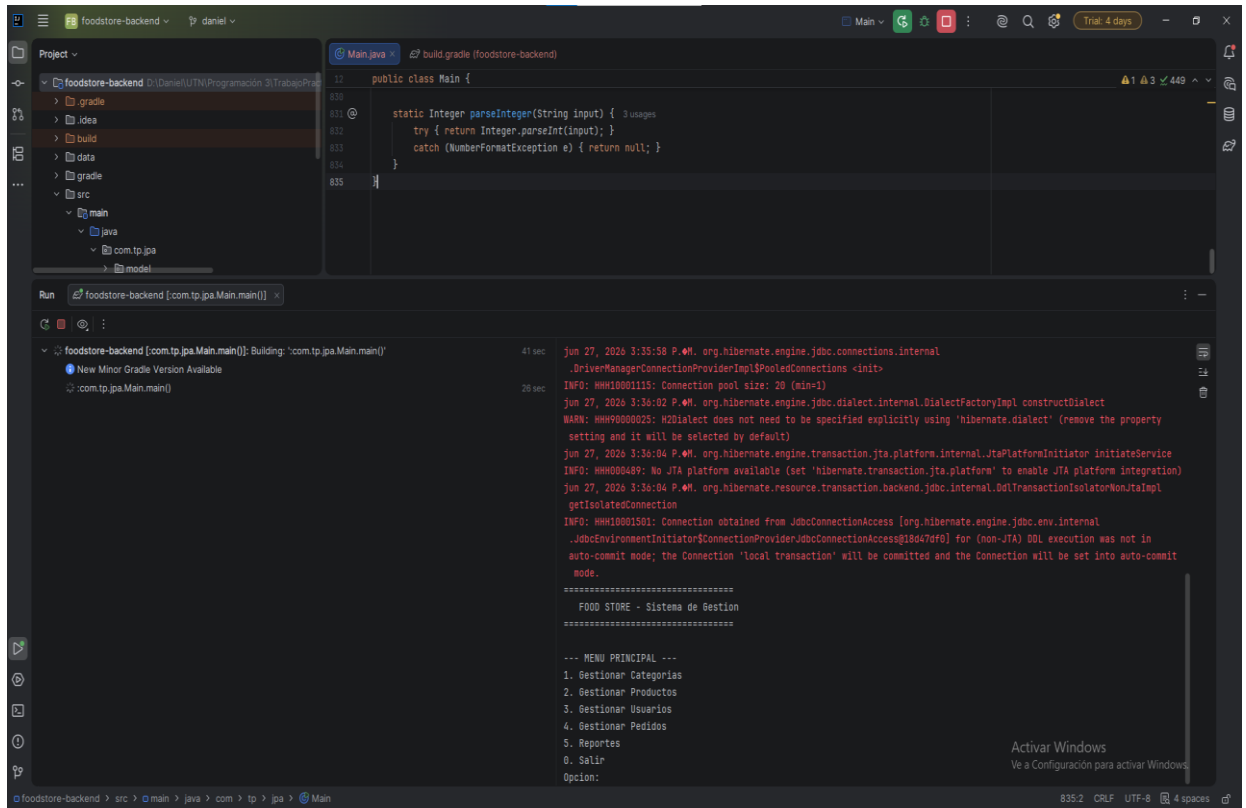
Ver Tienda

Gestión de Pedidos

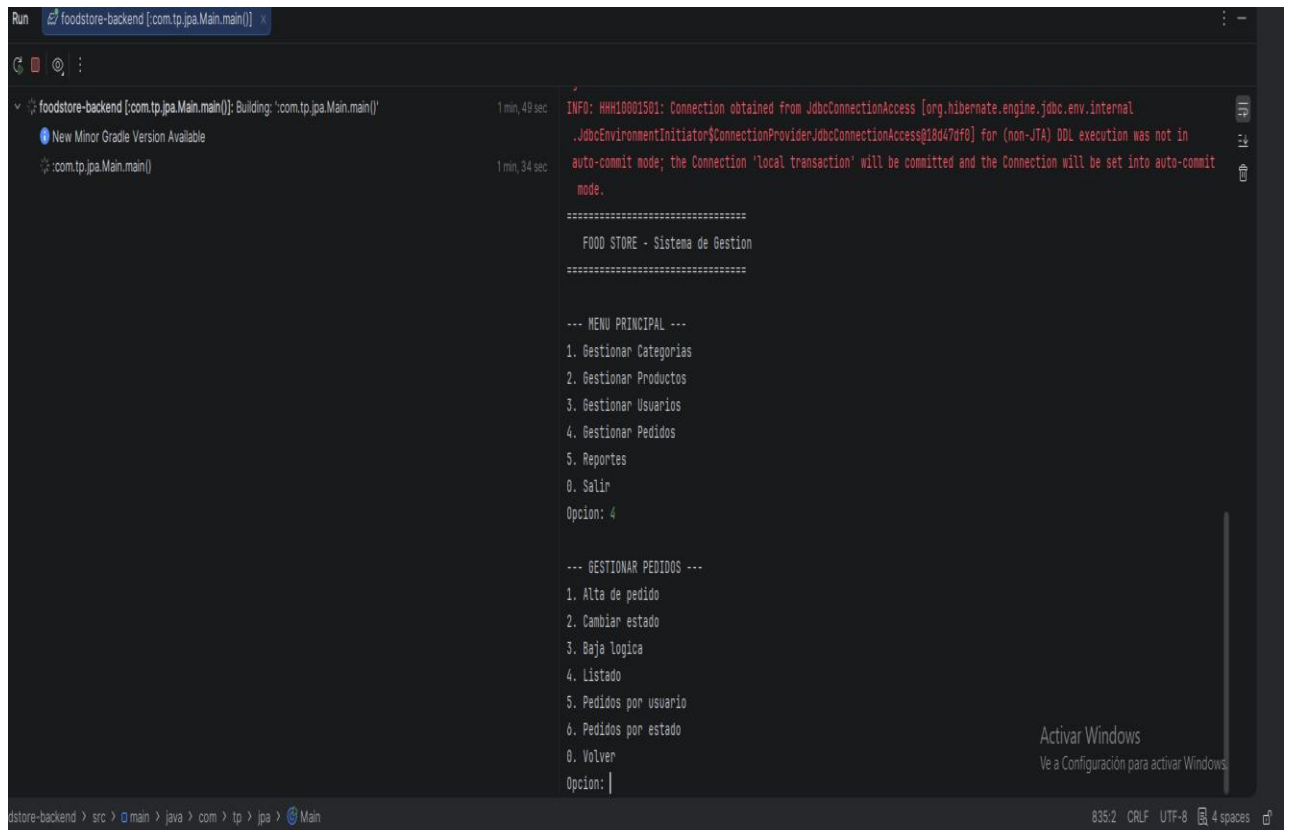
Todos los pedidos

Pedido #9 Cliente: Juan Pérez 2026-06-27 1 producto(s)	Pendiente \$4.500 Cambiar Estado
Pedido #8 Cliente: Juan Pérez 2026-06-26 1 producto(s)	Entregado \$5.000 Cambiar Estado
Pedido #4 Cliente: Admin 2026-06-25 1 producto(s)	Entregado \$4.500 Cambiar Estado
Pedido #5 Cliente: Admin 2026-06-25 1 producto(s)	Entregado \$13.500 Cambiar Estado
Pedido #6 Cliente: Admin 2026-06-25 1 producto(s)	Entregado \$6.200 Cambiar Estado
Pedido #7 Cliente: Juan Pérez 2026-06-25 1 producto(s)	Entregado \$7.600 Cambiar Estado

- Menú de consola Backend — captura del menú principal corriendo en IntelliJ IDEA



- Alta de pedido Backend — captura del flujo de creación de pedido en consola



```
Run foodstore-backend [com.tp.jpa.Main.main()] x
1 min, 49 sec INFO: HHK18001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal
JdbcEnvironmentInitiator$ConnectionProviderJdbcConnectionAccess@18d47df0] for (non-JTA) DDL execution was not in
auto-commit mode; the Connection 'local transaction' will be committed and the Connection will be set into auto-commit
mode.

=====
FOOD STORE - Sistema de Gestion
=====

--- MENU PRINCIPAL ---
1. Gestionar Categorías
2. Gestionar Productos
3. Gestionar Usuarios
4. Gestionar Pedidos
5. Reportes
0. Salir
Opcion: 4

--- GESTIONAR PEDIDOS ---
1. Alta de pedido
2. Cambiar estado
3. Baja logica
4. Listado
5. Pedidos por usuario
6. Pedidos por estado
0. Volver
Opcion: |

Activar Windows
Vé a Configuración para activar Windows

foodstore-backend > src > main > java > com > tp > jpa > Main 835/2 CRLF UTF-8 4 spaces
```

6. DIFICULTADES Y SOLUCIONES

LazyInitializationException en Alta de Producto

Durante el desarrollo del backend, al intentar agregar un producto a una categoría, se produjo una `LazyInitializationException` de Hibernate. El error ocurría porque se intentaba acceder a la colección lazy productos de `Categoria` después de que el `EntityManager` ya había sido cerrado.

Solución: Se refactorizó el método `altaProducto()` para manejar toda la operación dentro de un único `EntityManager` abierto explícitamente. Primero se persiste el producto con `em.persist()`, luego se recupera la categoría con `em.find()` dentro del mismo contexto de persistencia, y finalmente se agrega el producto a la colección. Esto garantiza que la colección lazy puede ser inicializada correctamente.

Conflicto de Puertos entre Live Server y Vite

Al intentar abrir los archivos HTML del frontend directamente con la extensión Live Server de VS Code, el navegador mostraba errores de MIME type porque intentaba cargar archivos `.ts` como si fueran scripts JavaScript comunes.

Solución: Se estableció como regla de trabajo acceder siempre al frontend a través del servidor de Vite (`http://localhost:5173`), que se encarga de compilar TypeScript en tiempo real. Live Server fue desactivado para este proyecto.

Persistencia de Cambios del Admin visible por el Cliente

Al principio, los cambios realizados por el administrador (cambio de estado de pedidos, edición de productos) no eran visibles para el cliente porque cada módulo leía directamente desde los archivos JSON originales.

Solución: Se implementó una estrategia de persistencia con `localStorage`: todos los módulos (admin y cliente) leen primero desde `localStorage` y, si no existe data guardada, usan el JSON como fallback. Cuando el admin

realiza cambios, los guarda en localStorage, y cuando el cliente consulta sus pedidos o el catálogo, obtiene los datos actualizados.

Codificación de Caracteres en la Consola de Windows

Al ejecutar el backend en la consola de Windows, los caracteres especiales del español (acentos, ñ) se mostraban como caracteres extraños debido a diferencias en la codificación de caracteres entre la JVM y la consola de Windows.

Solución: Se reemplazaron todos los textos del menú de consola por versiones sin acentos ni caracteres especiales (por ejemplo, "Gestionar Categorías" pasó a "Gestionar Categorias"), eliminando el problema de visualización sin afectar la funcionalidad del sistema.

7. BIBLIOGRAFÍA / WEBGRAFÍA

- Documentación oficial de Jakarta Persistence:
<https://jakarta.ee/specifications/persistence/>
- Documentación oficial de Hibernate ORM:
<https://hibernate.org/orm/documentation/>
- Documentación oficial de H2 Database:
<https://h2database.com/html/main.html>
- Documentación oficial de Vite: <https://vitejs.dev/guide/>
- Documentación oficial de TypeScript:
<https://www.typescriptlang.org/docs/>
- Repositorio oficial de Gradle: <https://gradle.org/docs/>
- MDN Web Docs - localStorage:
<https://developer.mozilla.org/es/docs/Web/API/Window/localStorage>
- MDN Web Docs - HTML5:
<https://developer.mozilla.org/es/docs/Web/HTML>
- Oracle Java 23 Documentation:
<https://docs.oracle.com/en/java/javase/23/>